

Lecture 11: Classification metrics, trees

Max G'Sell
Machine Learning I: 46-927

December 7, 2020

Announcements

- ▶ There will be six homeworks and six quizzes.

Today

Today we will talk more about classification metrics, and then potentially introduce generative models.

- ▶ Quick quiz discussion
- ▶ Classification:
 - ▶ Classification metrics
- ▶ Maybe a bit of decision trees

Classification so far

Focus on binary classification,

- ▶ $Y = 1$ if an event of interest happened
- ▶ $Y = 0$ if it did not happen

Most of our classifiers will give us a guess $\hat{p}(x) = \hat{\mathbb{P}}(Y = 1|X = x)$.
This is an estimate of the true $p(x) = \mathbb{P}(Y = 1|X = x)$.

We saw that to minimize average misclassification error,
 $\mathbb{E}\mathbf{1}_{f(X) \neq Y}$, you should choose $f(x) = \mathbf{1}_{p(x) > 1/2}$.

Let's think about this more carefully.

Confusion matrix: Example

For the homework problem, $Y = 1$ if an individual responds to marketing. The classification rule $\hat{p}(x) > 0.5$ gives a confusion matrix similar to:

	True $Y = 1$	True $Y = 0$
Guess $Y = 1$	0	2
Guess $Y = 0$	1083	7957

Confusion matrix: error rates

	True $Y = 1$	True $Y = 0$
Guess $Y = 1$	True Positive	False Positive
Guess $Y = 0$	False Negative	True Negative

Other error rate measures better capture asymmetric concerns when classifying:

Sensitivity: Fraction of points with $Y = 1$ that we find.

- Want high Sensitivity when False Negatives are more costly than False Positives. Example: Fraud detection.

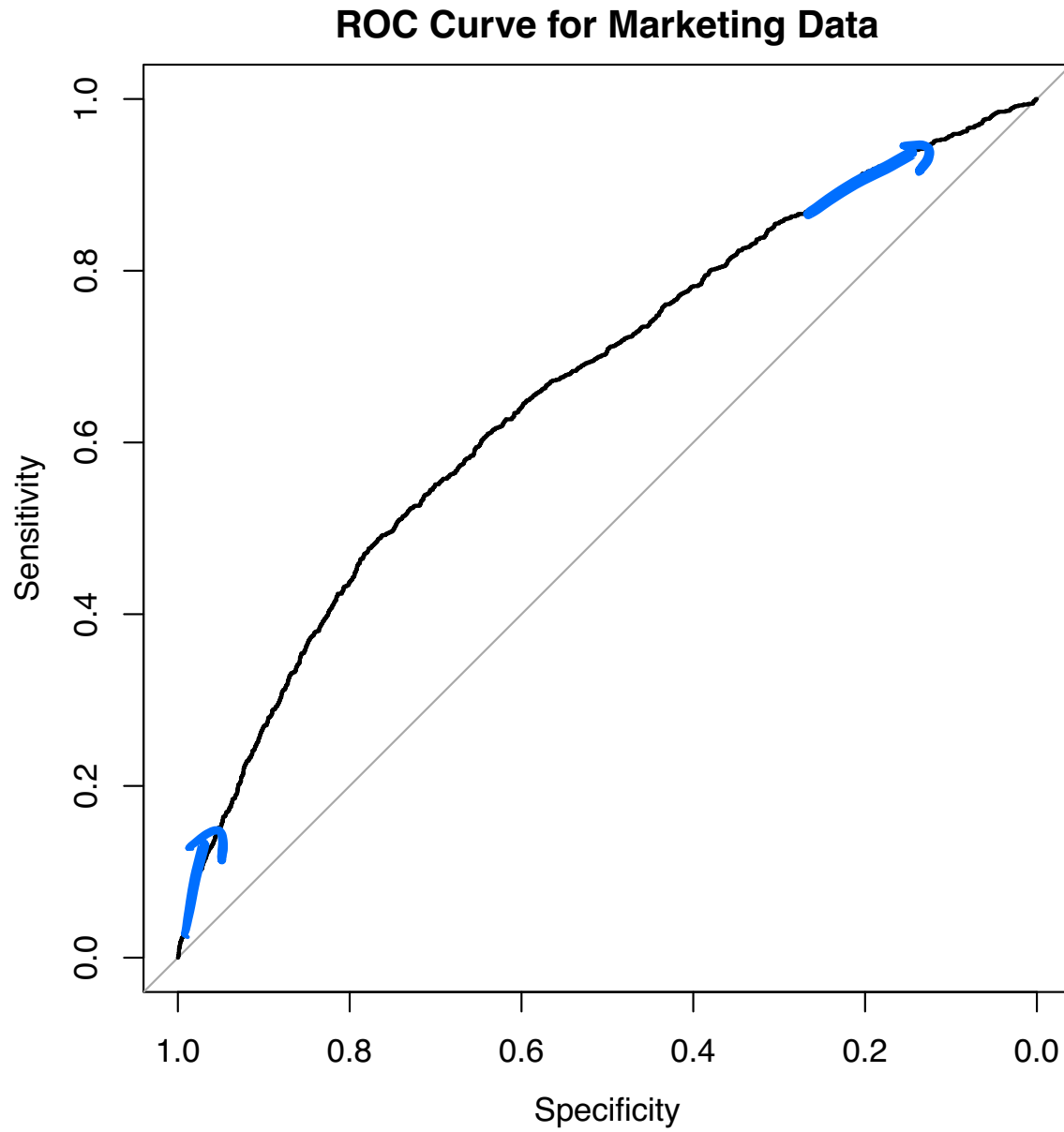
Specificity: Fraction of points with $Y = 0$ that we avoid.

- Want high Specificity when False Positives are more expensive than False Negatives. Example: Criminal trials.

We would like both of them to be high, but we need to make tradeoffs as we adjust our threshold.

$$P(Y=1 | x=x)$$

ROC Curve



HW5
Problem 1.2

Other metrics

	True $Y = 1$	True $Y = 0$
Guess $Y = 1$	True Positive	False Positive
Guess $Y = 0$	False Negative	True Negative

Positive predictive value (PPV):

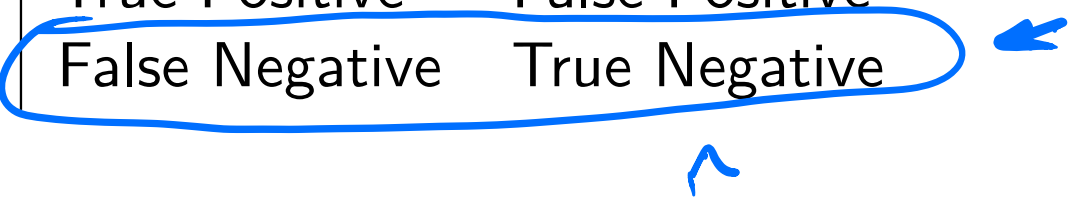
$$P(Y=1 \mid \text{Guess } Y=1)$$

$$\begin{aligned} PPV &= \frac{\# \text{ observations correctly classified as positive}}{\# \text{ observations classified as positive}} \\ &= \frac{TP}{TP + FP} \end{aligned}$$

Suppose that you take a test for a disease. The PPV captures the probability that you actually have the disease when you test positive.

Other metrics

	True $Y = 1$	True $Y = 0$
Guess $Y = 1$	True Positive	False Positive
Guess $Y = 0$	False Negative	True Negative



Negative Predictive Value (NPV):

$$\begin{aligned} NPV &= \frac{\# \text{ observations correctly classified as negative}}{\# \text{ observations classified as negative}} \\ &= \frac{TN}{TN + FN} \end{aligned}$$

Suppose that you take a test for a disease. The NPV is the probability that you are not sick, given that the test comes up negative.

Calibration

Is our $\hat{p}(x)$ a good estimate of $p(x) = \mathbb{P}(Y = 1|X = x)$?

Why do we care? We're just going to threshold it...

There is information in knowing $\mathbb{P}(Y = 1|X = x)$.

- ▶ A case that is definitely fraud ($p(x) = 0.99$) may be handled very differently than a case that might be fraud ($p(x) = 0.52$).

Calculating the proper trade-offs for making decisions depends on the probabilities, not just the categories.

Note: Even if $\hat{p}(x)$ is not a good approximation of $p(x)$, thresholding $\hat{p}(x)$ might still give good guess at categories.

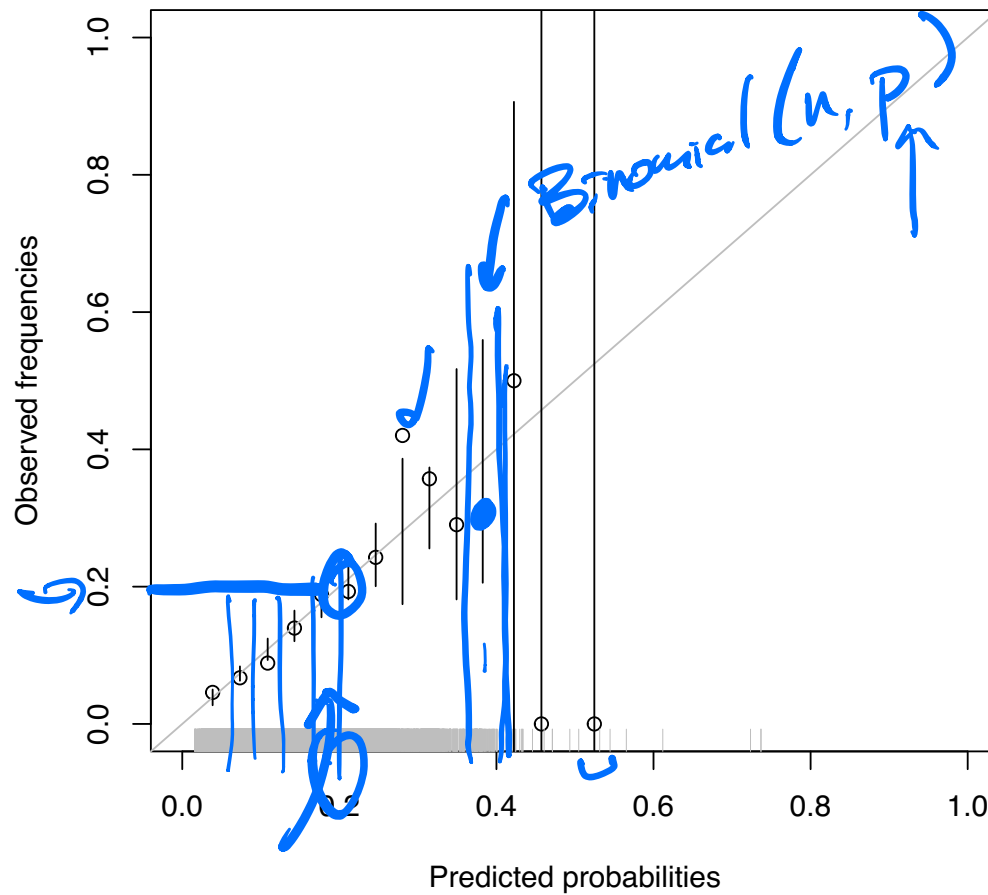
Calibration plots

To check how well calibrated your $\hat{p}(x)$ is, you can make a *calibration plot*:

1. Bin the data according to predicted probability, $\hat{p}(x)$. E.g., $[0, 0.1]$, $(0.1, 0.2]$, $\dots$, $(0.9, 1]$.
2. For each bin, calculate the proportion of observations with class $Y = 1$
3. Plot these true fractions against the midpoints of the bin predicted probabilities

Calibration plots

Calibration plot



$$\hat{p}(x) \approx .4$$

Calibration plots

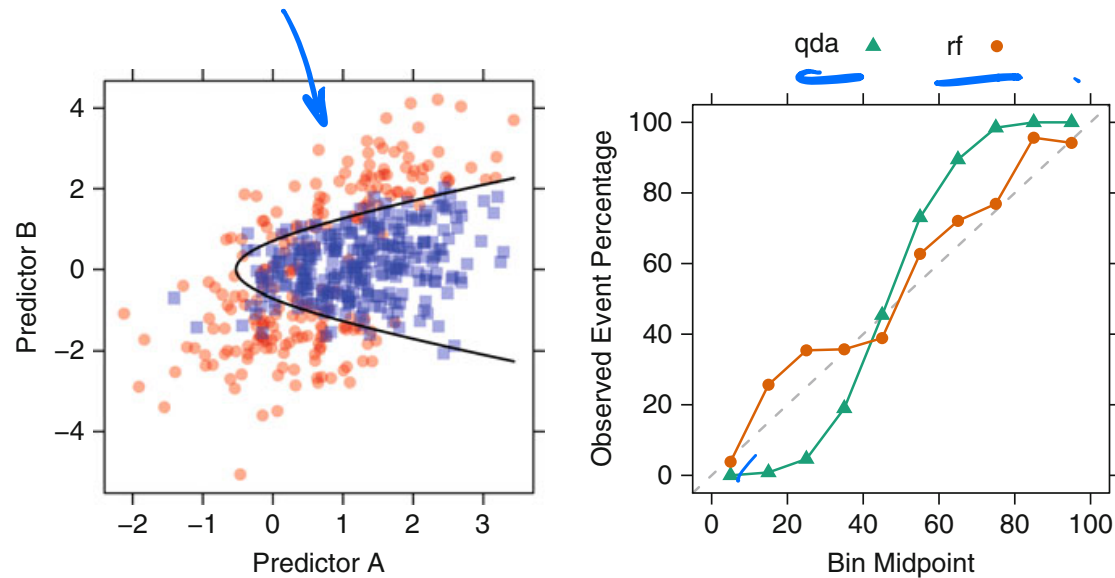


Fig. 11.1: *Left*: A simulated two-class data set with two predictors. The *solid black line* denotes the 50% probability contour. *Right*: A calibration plot of the test set probabilities for random forest and quadratic discriminant analysis models

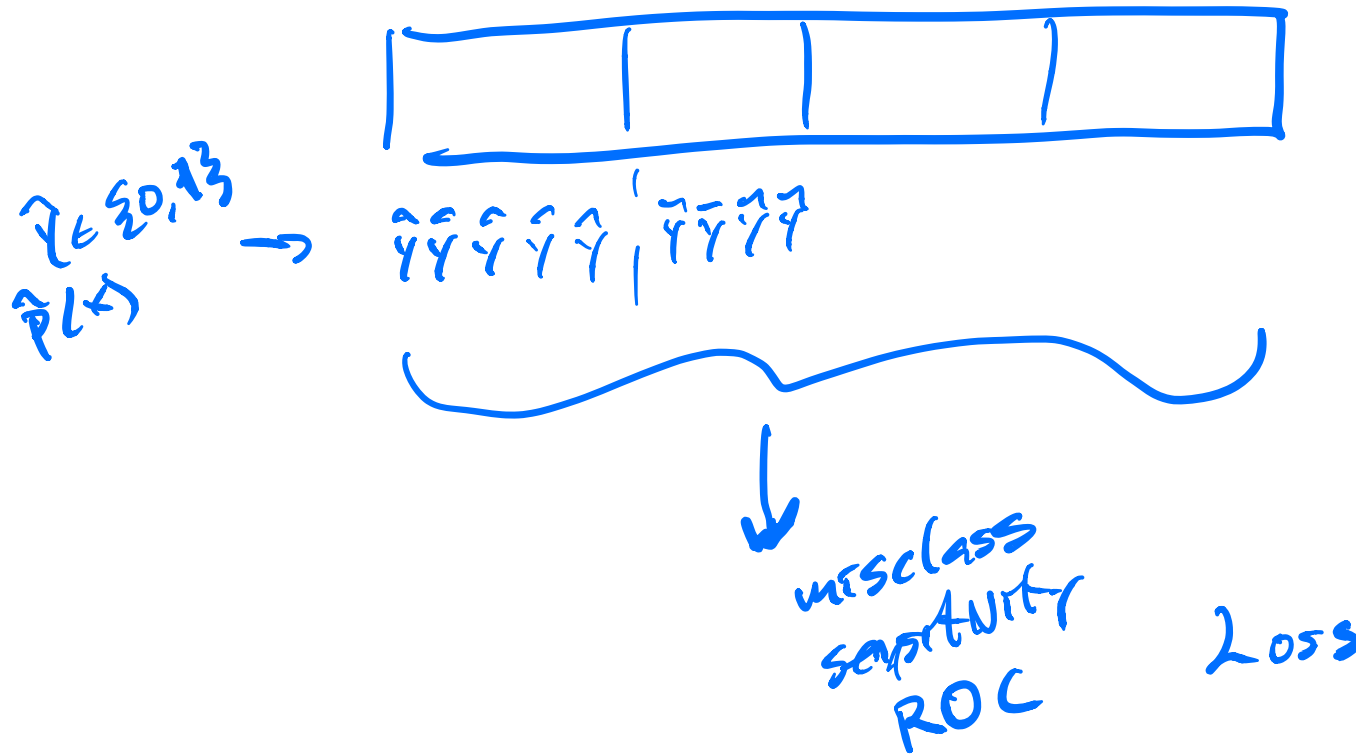
(From Applied Predictive Modeling by Kjell Johnson and Max Kuhn)

If the calibration is off, you can try transforming your $\hat{p}(x)$.

$$\text{logit } \hat{p}_{\text{new}}^{\text{cal}} = f(\text{logit } \hat{p}(x))$$

Remark: Model validation and tuning

Suppose that we want to pick between different classification models. Or instead, that we need to choose λ in logistic lasso. How can we do this?



Where are we headed next?

We showed that the optimal misclassification error is achieved by either of the following ideal classifiers:

$$\begin{aligned}\hat{f}(x) &= \operatorname{argmax}_{j=1,\dots,K} \mathbb{P}(Y = j | X = x) \\ &= \operatorname{argmax}_{j=1,\dots,K} \mathbb{P}(X = x | Y = j) \cdot \pi_j\end{aligned}$$

1. **Discriminative models:** We can try to estimate $\mathbb{P}(Y = j | X = x)$ directly.

► Continue here with more complex models

2. **Generative models:** We could try to estimate the distributions $\mathbb{P}(X = x | Y = j)$ for each class.

► Deferring until later / next mini

Logistic regression

We introduced logistic regression as a first example of a discriminative classifier.

We model $Y_i \sim \text{Bernoulli}(P(Y = 1|X = x))$ with

$$P(Y = 1|X = x) = \frac{e^{\beta^T x}}{1 + e^{\beta^T x}} = \frac{1}{1 + e^{-\beta^T x}}$$

and then maximize the likelihood.

However, these modeling choices can be quite restrictive. Even if we expand beyond simple linear functions, we have to specify a good functional form and the important interactions.

We will now construct an alternative set of discriminative classifiers that will circumvent these problems.

Flexible models

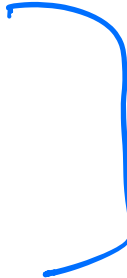
We've seen a couple flexible models:

- ▶ Additive models: flexible about transformations, but require interactions to be specified.
- ▶ k-nearest neighbors: can be flexible and predict well, with enough data and a good distance. However, nearly impossible to interpret! No simplification, requires all training data to make a prediction. Struggles in high dimensions.
- ▶ SVMs: (In the future, ML 2) Can capture interactions or transformations indirectly by changing kernels, but the connections are not direct and only some can be captured for a given kernel.

Today we are going to introduce new styles of flexible methods that avoid many of these problems.

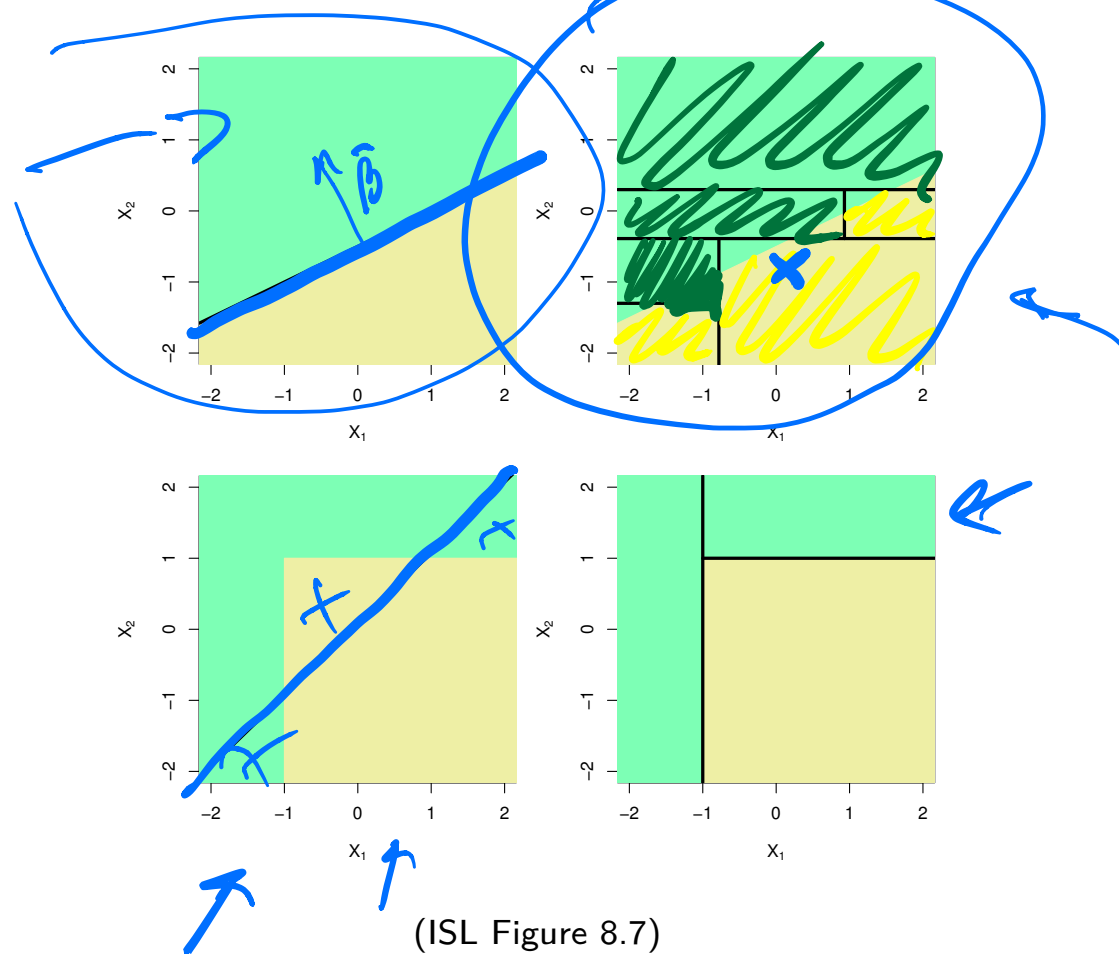
Wish list

There are many properties that we might want for a good prediction method.

- ▶ Good predictions and good probability estimates
 - ▶ The ability to tune bias/variance
 - ▶ Interpretability: What are our predictions based on? 
 - ▶ Automatic handling of variable transformations
 - ▶ Automatic detection of interactions
- 

Overview: Tree-based methods

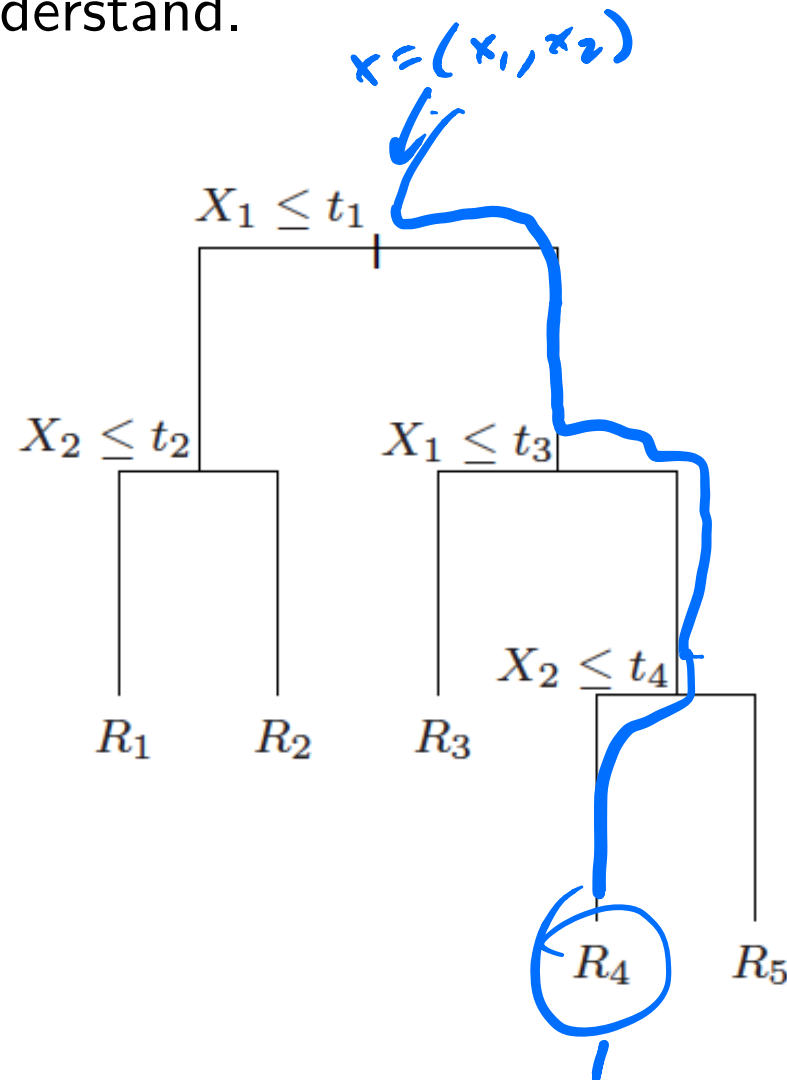
Tree-based methods for predicting y from a feature vector $x \in \mathbb{R}^p$ divide up the feature space into rectangles, and then fit a very simple model in each rectangle. This works both when y is discrete and continuous, i.e., both for **classification** and **regression**



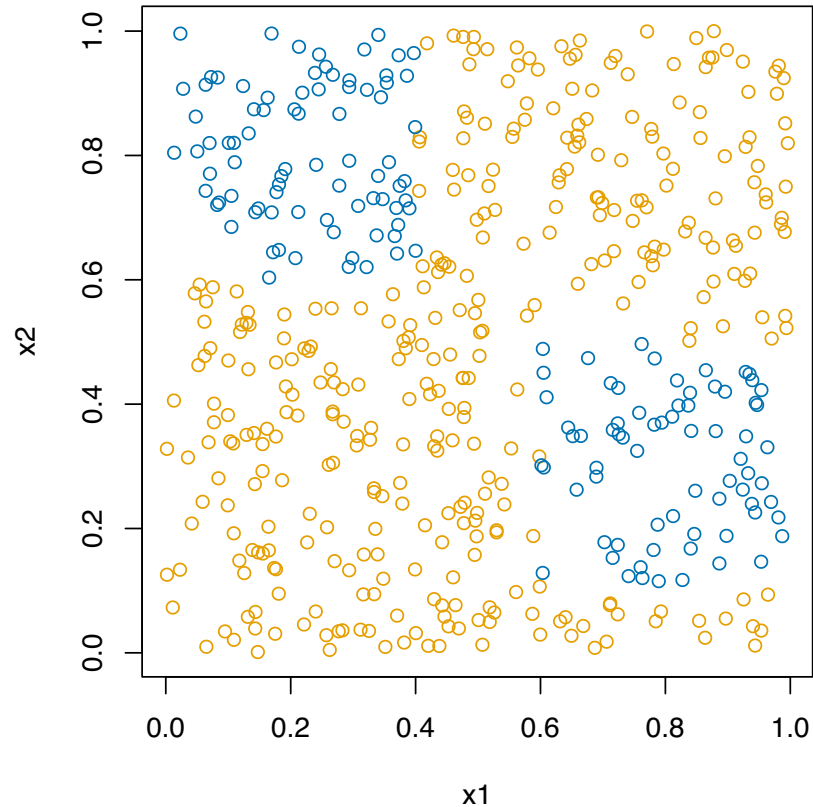
This is a big shift from thinking about linear-style models!

Tree-based methods

Tree-based methods will fix some of the problems we have encountered with other methods. They will give us a flexible rule, but one we can understand.



Overview: Tree-based methods

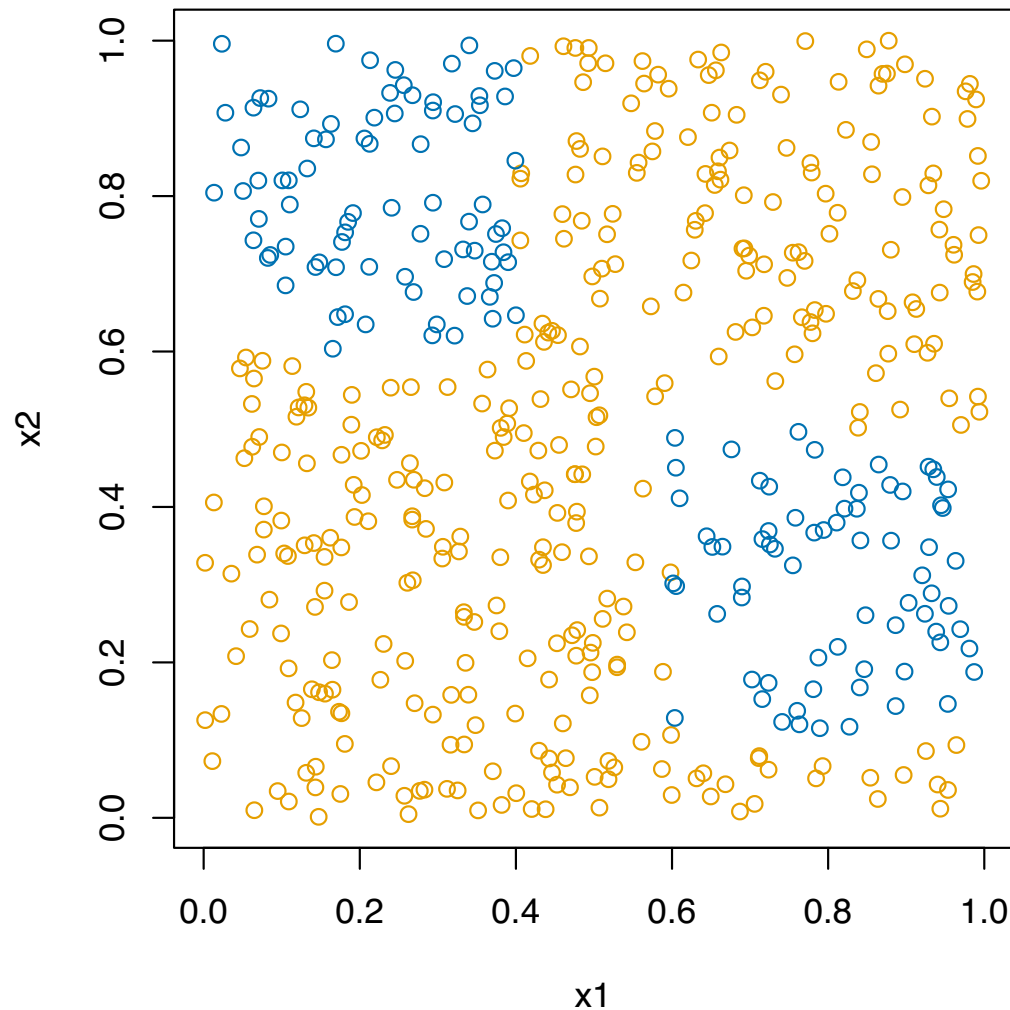


Does dividing up the feature space into rectangles look like it would work here?

Overview: Tree-based methods

Searching over all possible rectangular partitions is hard! There are many such partitions!

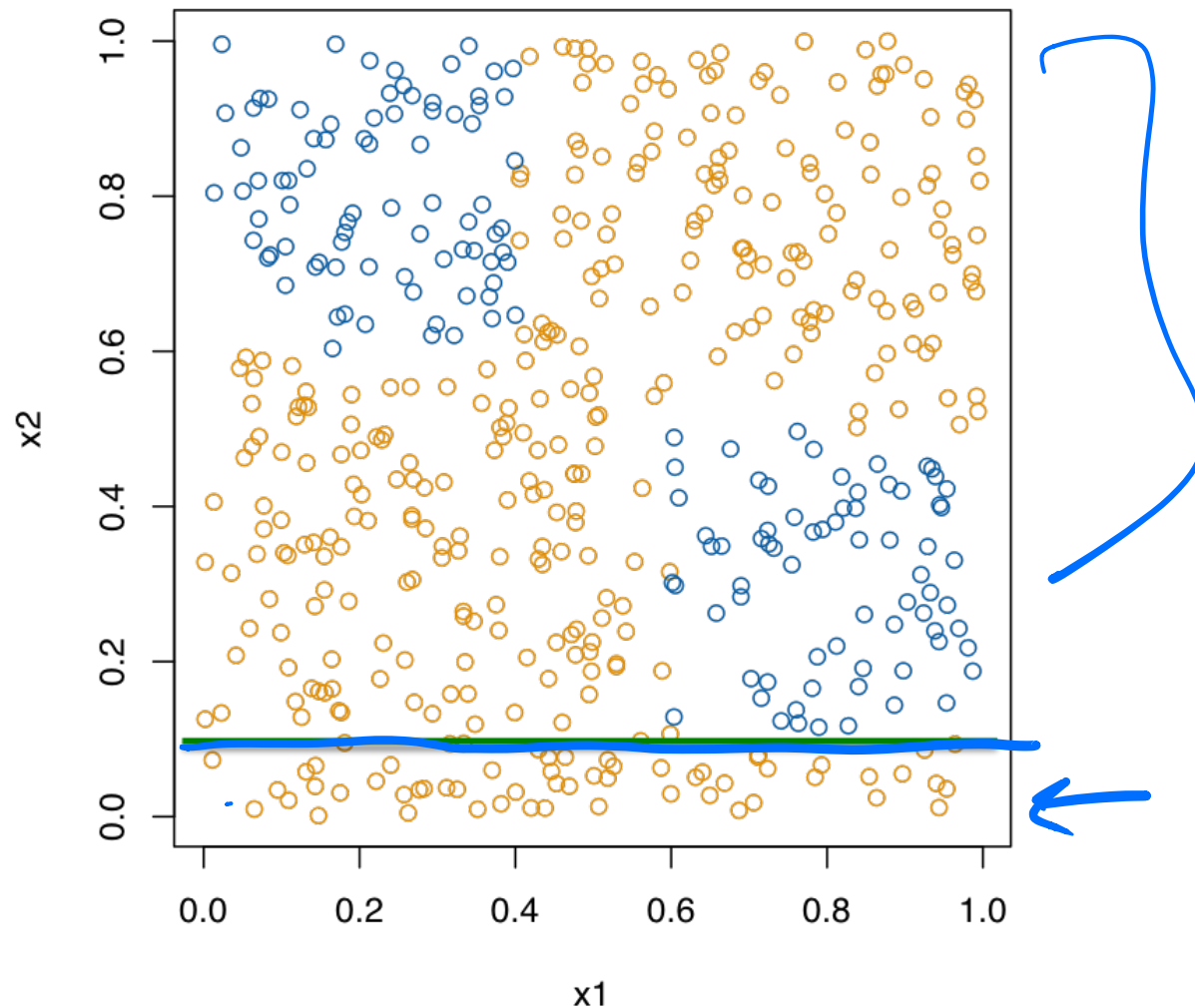
We make it simpler by considering a sequence of binary splits on individual variables.



Overview: Tree-based methods

Searching over all possible rectangular partitions is hard! There are many such partitions!

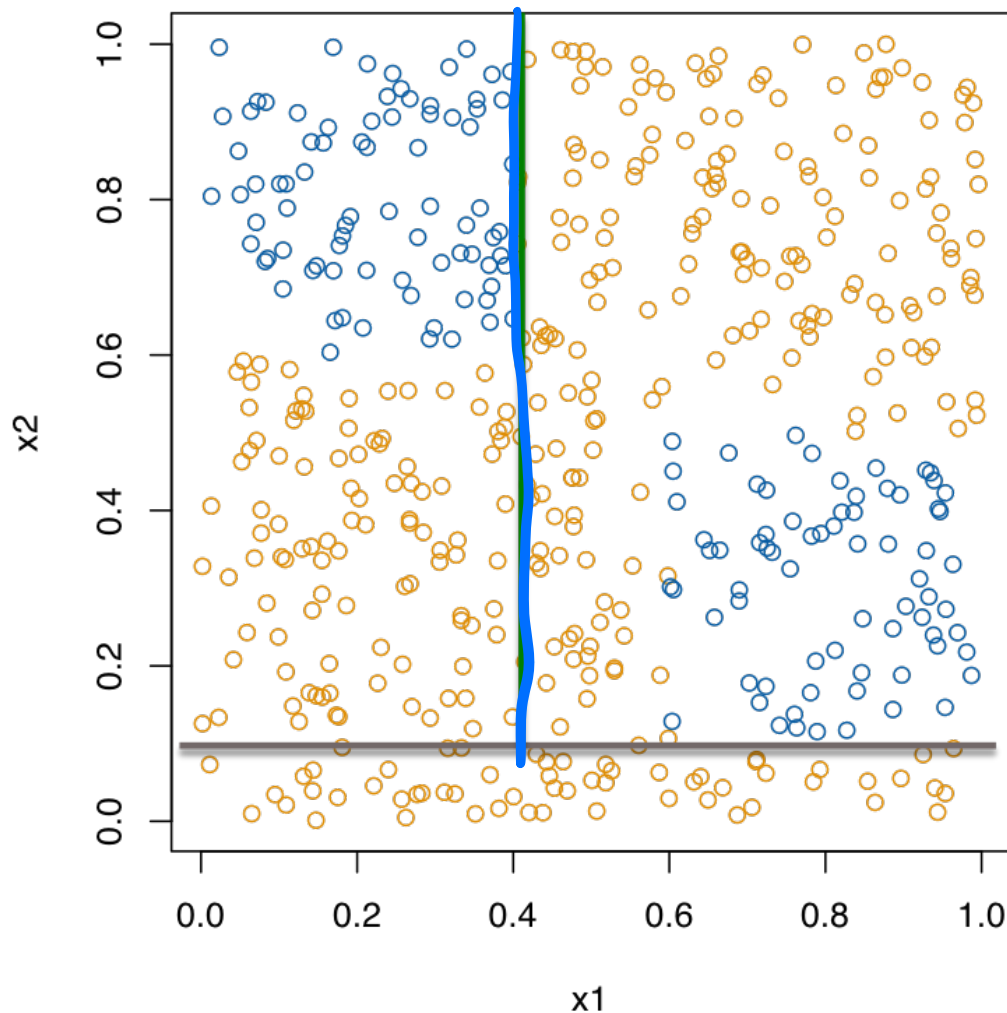
We make it simpler by considering a sequence of binary splits on individual variables.



Overview: Tree-based methods

Searching over all possible rectangular partitions is hard! There are many such partitions!

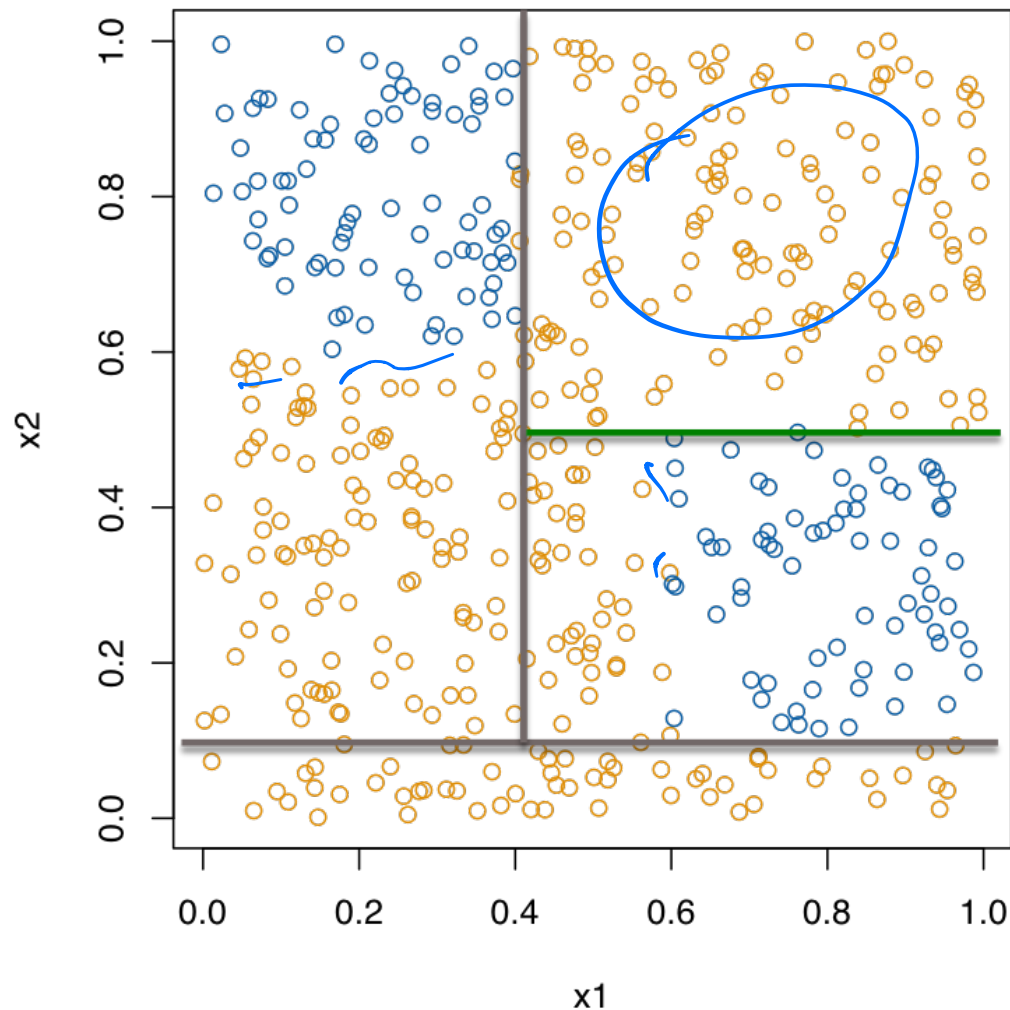
We make it simpler by considering a sequence of binary splits on individual variables.



Overview: Tree-based methods

Searching over all possible rectangular partitions is hard! There are many such partitions!

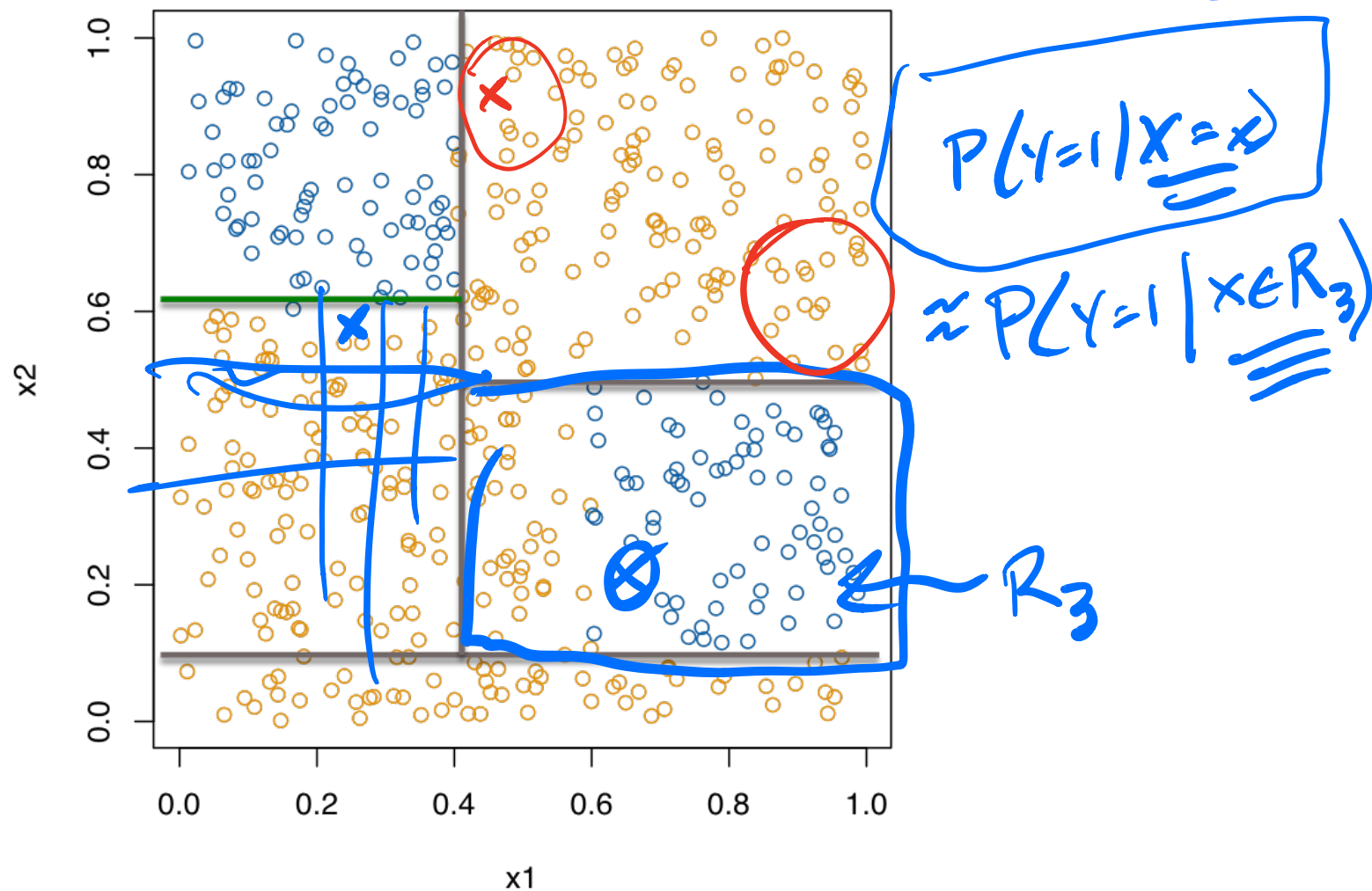
We make it simpler by considering a sequence of binary splits on individual variables.



Overview: Tree-based methods

Searching over all possible rectangular partitions is hard! There are many such partitions!

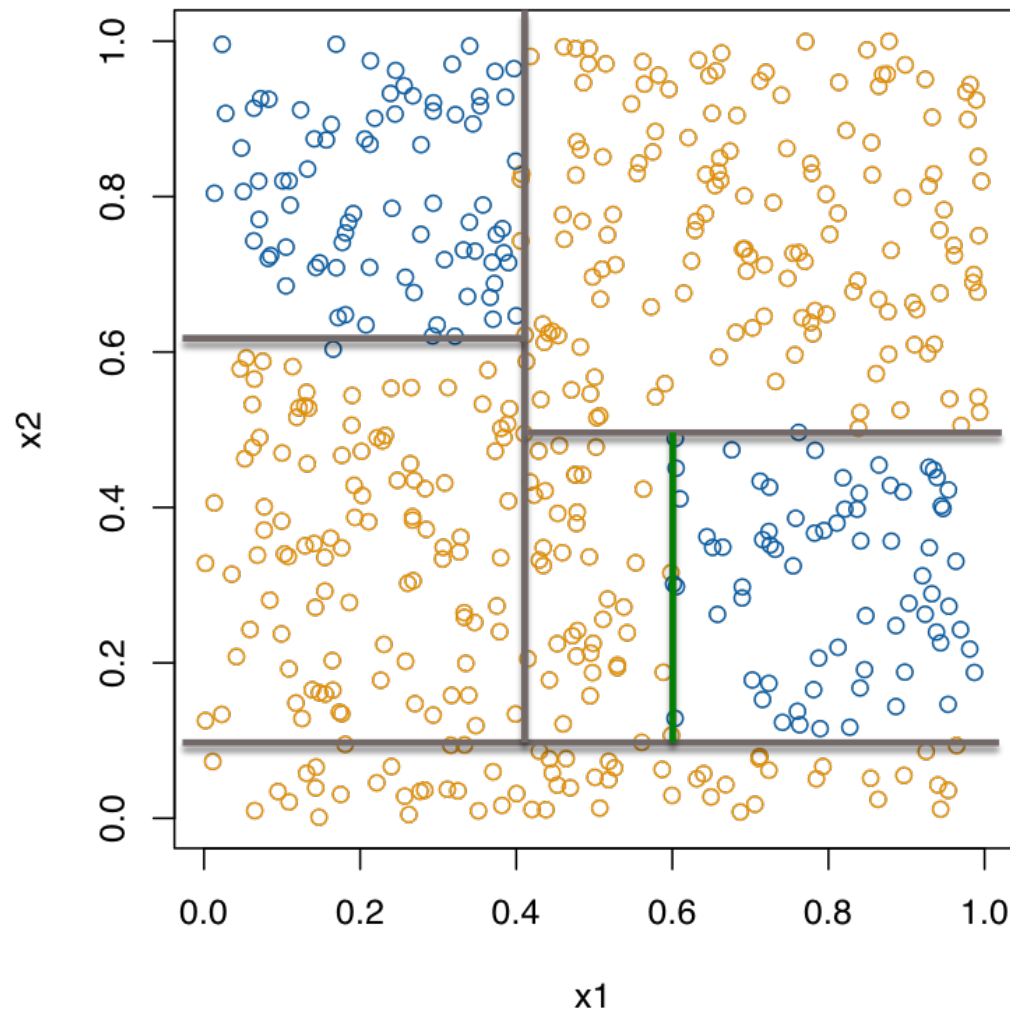
We make it simpler by considering a sequence of binary splits on individual variables.



Overview: Tree-based methods

Searching over all possible rectangular partitions is hard! There are many such partitions!

We make it simpler by considering a sequence of binary splits on individual variables.



Overview: Tree-based methods

We simplify our partition search by only considering a sequence of binary splits on single variables.

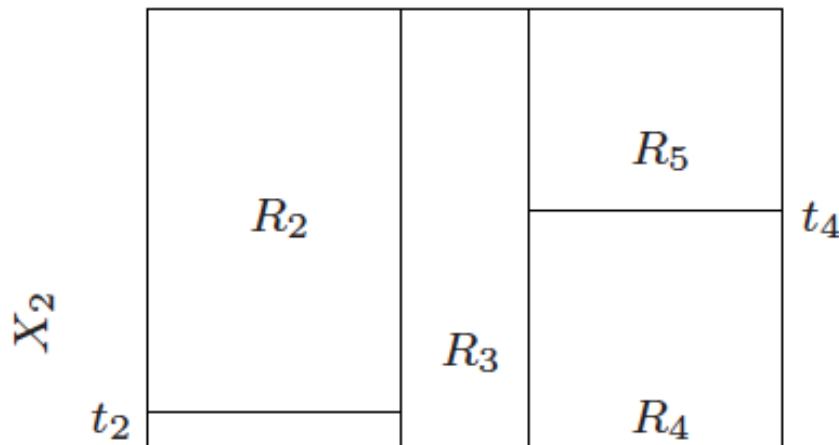
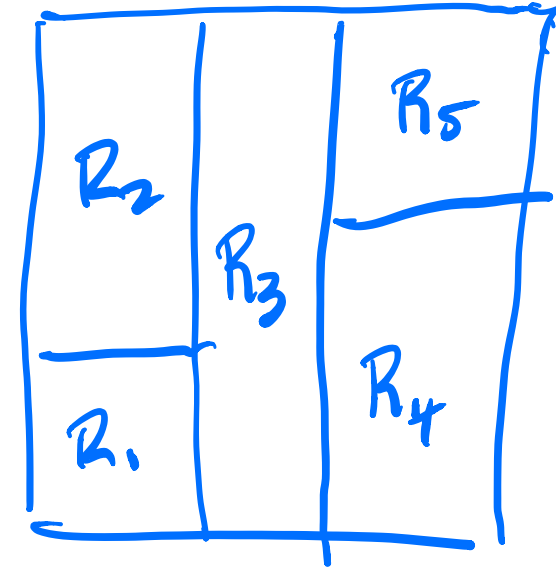
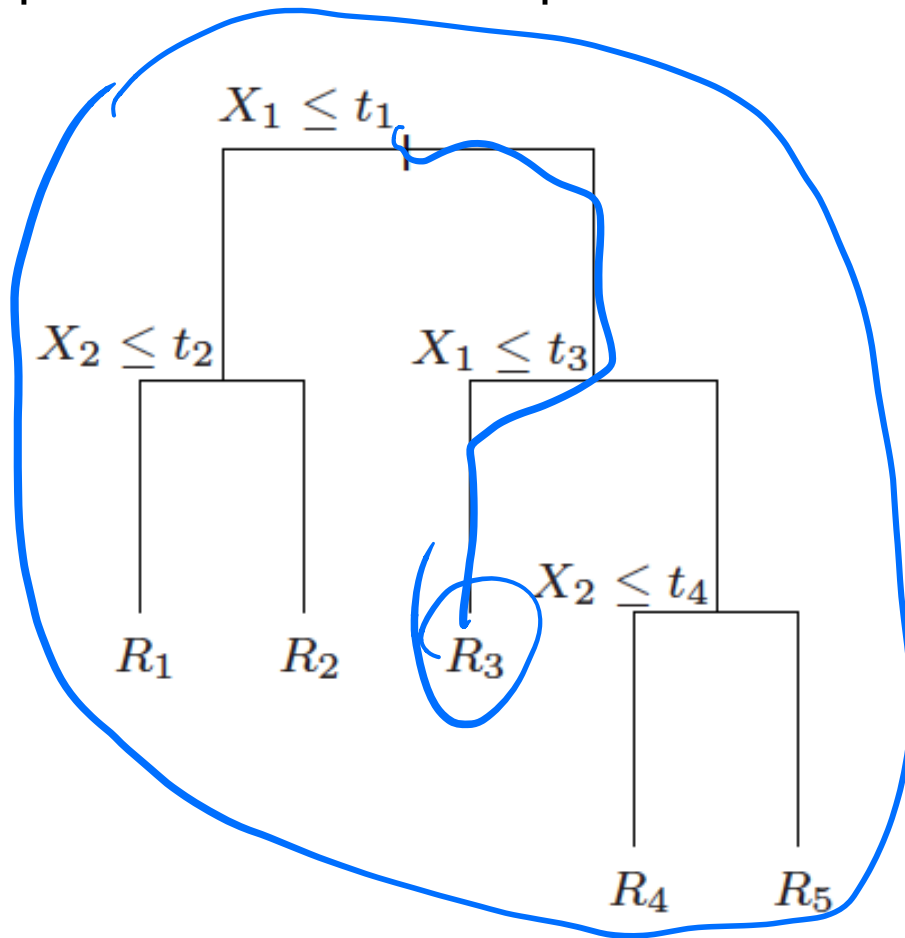
I.e., we choose a variable X_j , $j = 1, \dots, p$, **divide** up the feature space according to

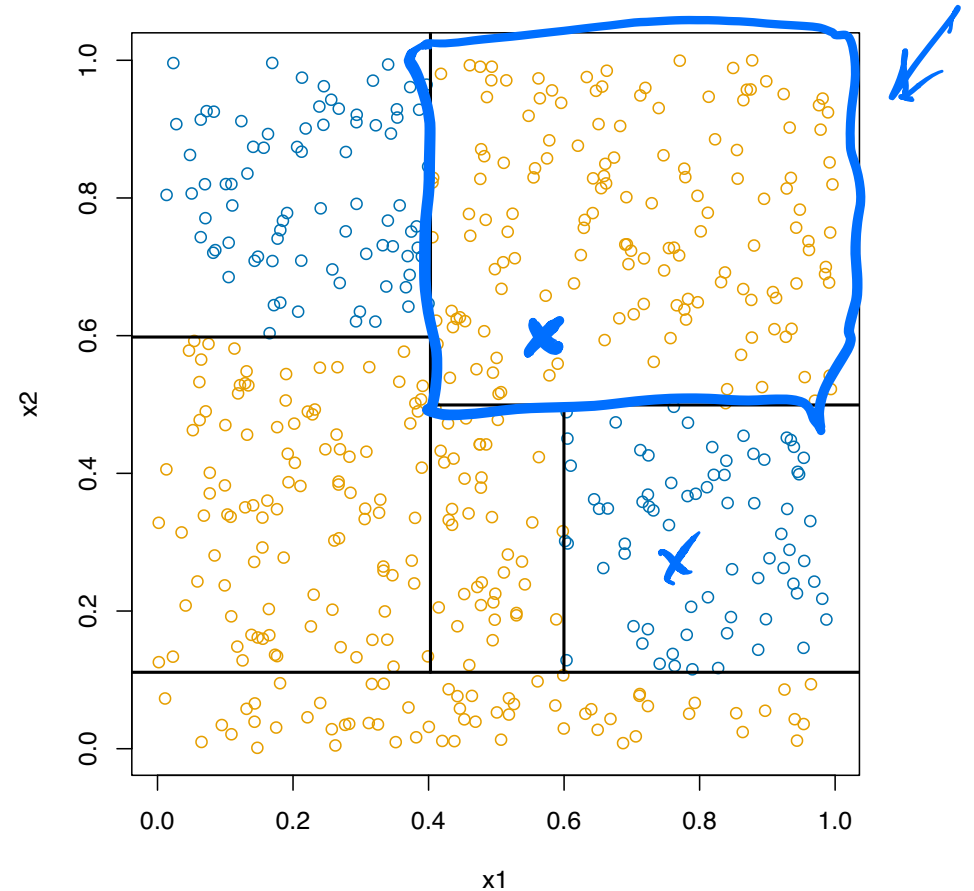
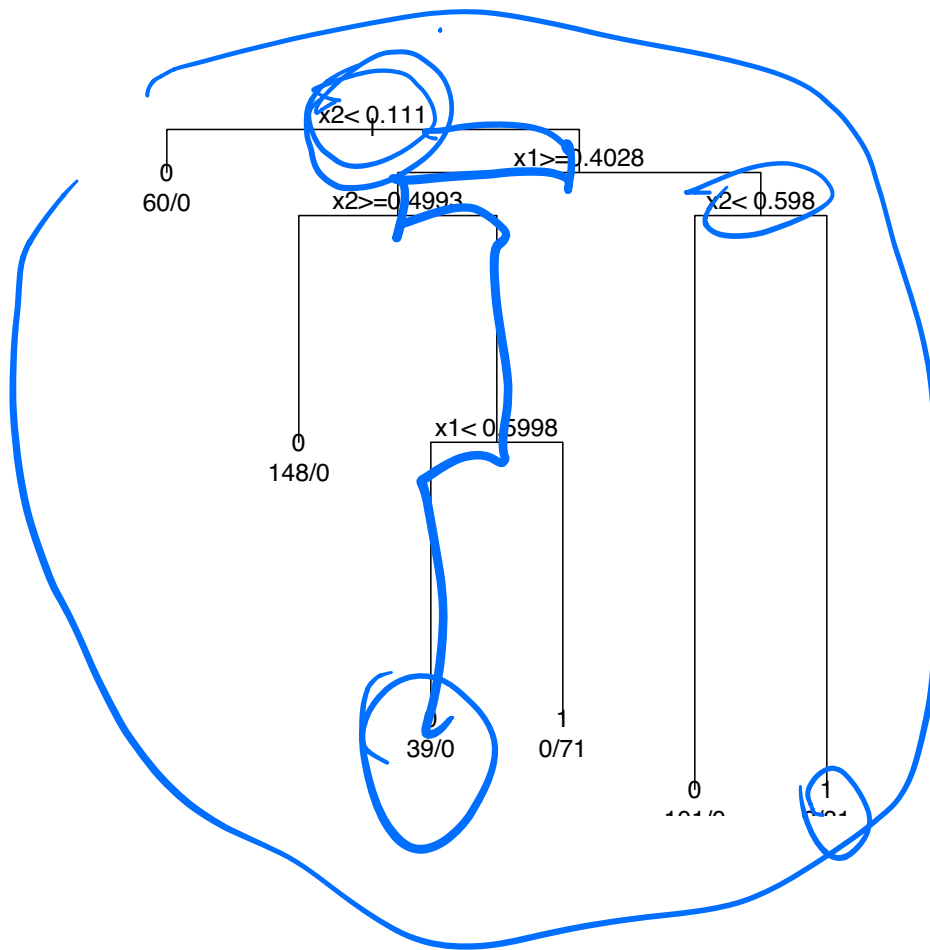
$$X_j \leq c \text{ and } X_j > c.$$

Then we proceed on each half separately.

To make this even simpler, we choose the splits in a “greedy” fashion, only looking at the payoff for the current split and ignoring advantages for future splits.

A benefit of this search approach is that it also gives a natural tree representation of the partition.





This gives a rule that is easy to understand, easy to explain, and easy to implement!

No more coefficients to interpret!

Classification trees

Let's start with classification.

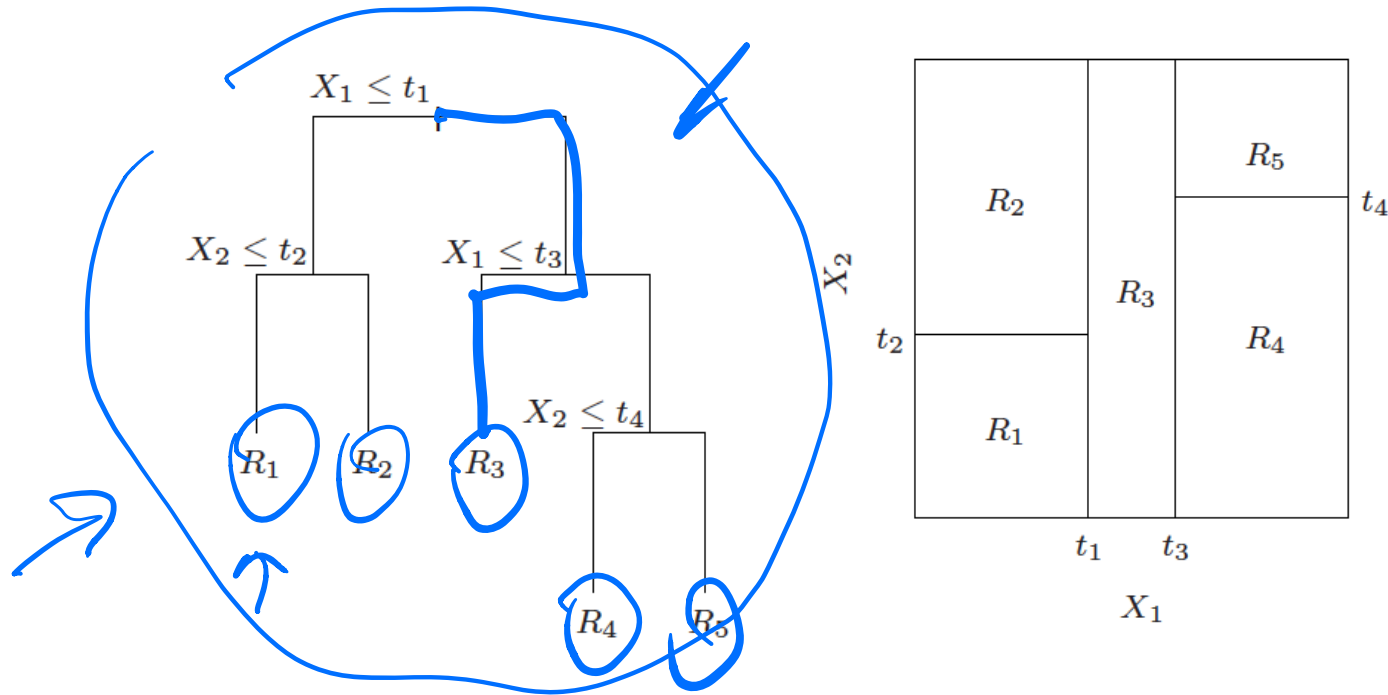
Classification trees are popular because they are interpretable, and sometimes because they mimic the way decisions are made.

Let $(x_i, y_i), i = 1, \dots, n$ be the training data, where $y_i \in \{1, \dots, K\}$ are the class labels, and $x_i \in \mathbb{R}^p$ measure the p predictor variables.

We are going to use a tree to define our classification function $\hat{f}(x) : \mathbb{R}^p \rightarrow \{1, \dots, K\}$.

Think about the tree taking the place of our linear model in previous methods.

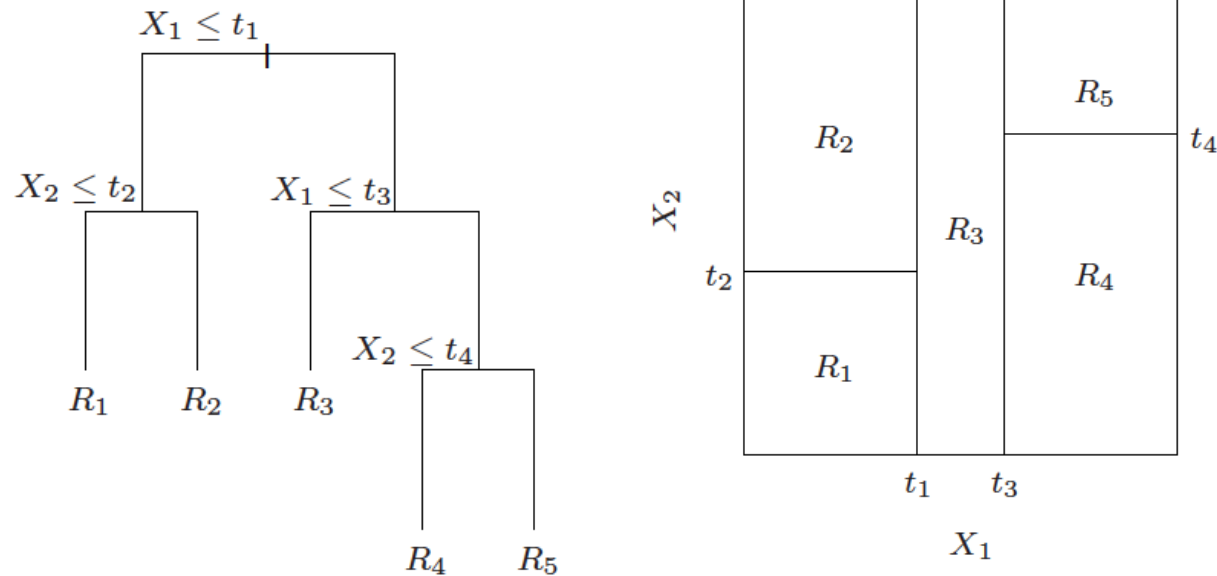
Classification trees



The classification tree can be thought of as defining m regions (rectangles) $R_1, \dots, R_m$, each corresponding to a leaf of the tree

We assign each R_j a class label $c_j \in \{1, \dots, K\}$. We then classify a new point $x \in \mathbb{R}^p$ by

$$\hat{f}^{\text{tree}}(x) = \sum_{j=1}^m c_j \cdot \mathbb{1}\{x \in R_j\} = c_j \text{ if } x \in R_j$$



$$\hat{f}^{\text{tree}}(x) = \sum_{j=1}^m c_j \cdot 1\{x \in R_j\}$$

Finding out which region a given point x belongs to is **easy** since the regions R_j are defined by a tree—we just scan down the tree. Otherwise, it would be a lot harder (need to look at each region)

Estimated class probabilities

We now have a flexible function that takes in covariates $x \in \mathbb{R}^p$ and produces classifications!

In the last few lectures, we realized that we want something more. We want actual estimates of the class probabilities!

How can we get these?

Estimated class probabilities

Note that each region R_j contains some subset of the training data (x_i, y_i) , $i = 1, \dots, n$, say, n_j points.

We have been predicting class c_j using the most common class among points in R_j .

For each class k , we can also estimate the probability that a point has that class, given that it falls in R_j , $P(C = k | X \in R_j)$, by

$$\hat{p}_k(R_j) = \frac{1}{n_j} \sum_{x_i \in R_j} 1\{y_i = k\}$$

the **proportion of points** in the region that are of class k .

We can even think of our predicted class as

$$c_j = \operatorname{argmax}_{k=1, \dots, K} \hat{p}_k(R_j)$$

How to build trees?

There are two main issues to consider in building a tree:

1. How to choose the splits?
2. How big to grow the tree?

Think first about varying the depth of the tree ... which is more complex/flexible, a big tree or a small tree? What **tradeoff** is at play here? How might we eventually consider choosing the depth?

Now for a fixed depth, consider choosing the splits. If the tree has depth d , it could have $\approx 2^d$ nodes. At each node we could choose any of p the variables for the split—this means that the number of possibilities is

$$p \cdot 2^d$$

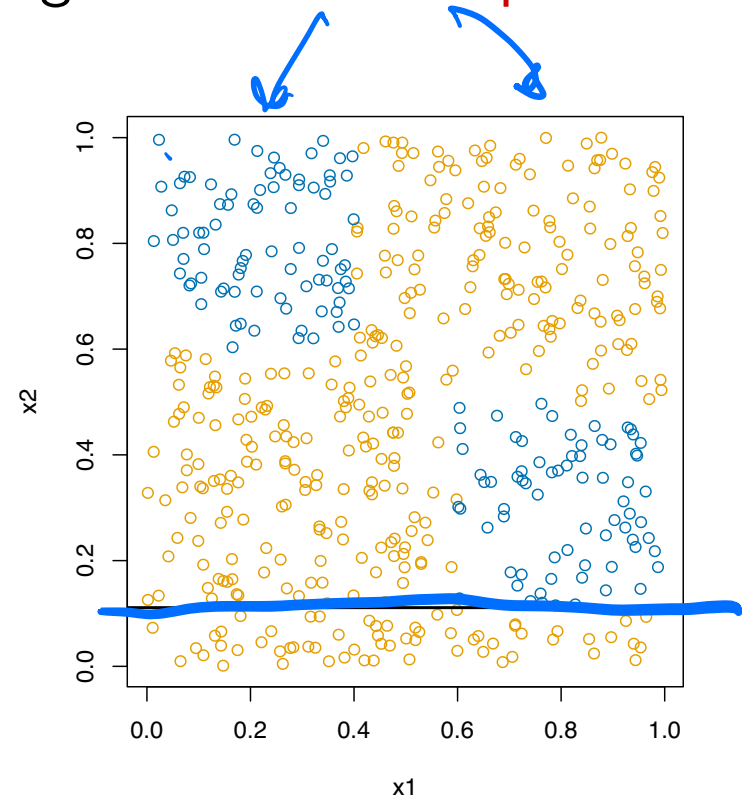
This is **huge** even for moderate d ! And we haven't even counted the actual split points themselves

The CART algorithm

The **CART algorithm**¹ chooses the splits in a top down fashion: First chooses the first variable to be the root, then the variables at the second level, etc.

At each stage we choose the split to achieve the biggest drop in misclassification error—this is called a **greedy** strategy. In terms of tree depth, the strategy is to grow a large tree and then **prune** at the end.

Why grow a large tree and prune, instead of just stopping at some point?



¹Breiman et al. (1984), "Classification and Regression Trees"

Recall that in a region R_m , the proportion of points in class k is

$$\hat{p}_k(R_m) = \frac{1}{n_m} \sum_{x_i \in R_m} 1\{y_i = k\}.$$

The CART algorithm begins by considering splitting on variable j and split point s , and defines the regions

$$R_1 = \{X \in \mathbb{R}^p : X_j \leq s\}, \quad R_2 = \{X \in \mathbb{R}^p : X_j > s\}$$

We then **greedily** chooses j, s by minimizing the misclassification error

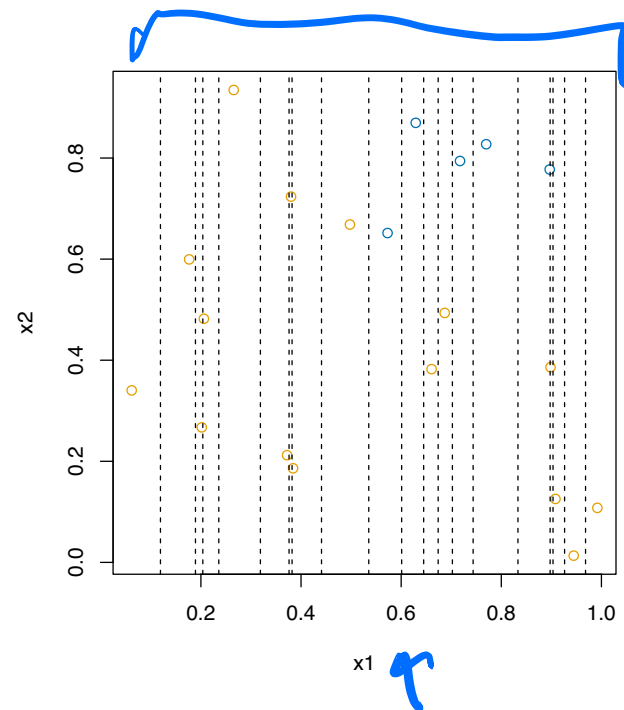
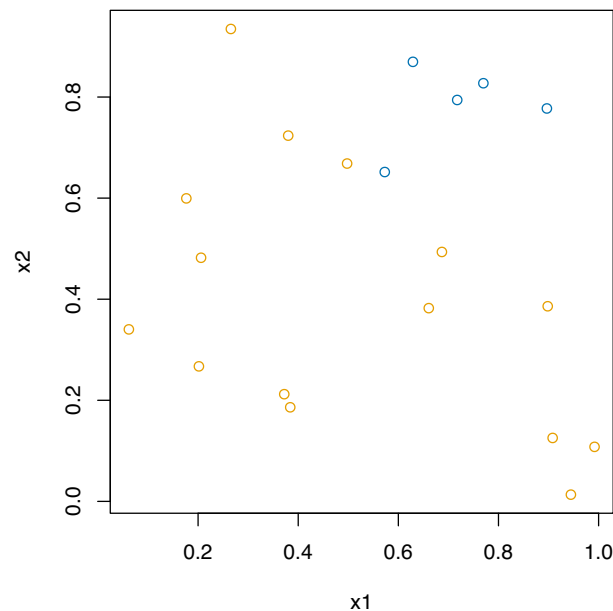
$$\operatorname{argmin}_{j, s} \left(n_{R_1} [1 - \hat{p}_{c_1}(R_1)] + n_{R_2} [1 - \hat{p}_{c_2}(R_2)] \right)$$

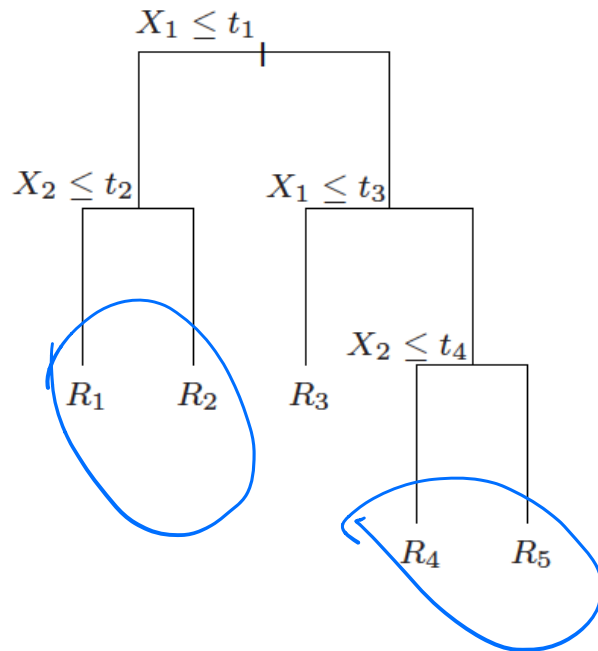
Here $c_1 = \operatorname{argmax}_{k=1, \dots, K} \hat{p}_k(R_1)$ is the most common class in R_1 , and $c_2 = \operatorname{argmax}_{k=1, \dots, K} \hat{p}_k(R_2)$ is the most common class in R_2

We now repeat this within each of the newly defined regions R_1, R_2 . Again consider all variables and split points for each of R_1, R_2 , greedily choosing the biggest improvement in misclassification error.

How

do we find the best split s ? Aren't there **infinitely many** to consider?



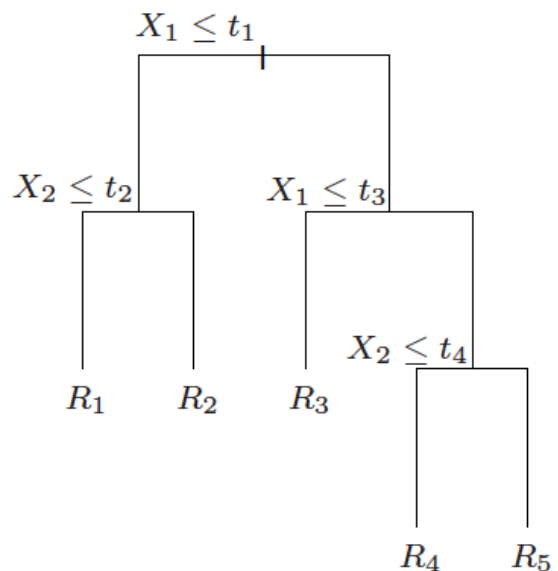


Continuing on in this manner, we will get a big tree T_0 . Its leaves define regions $R_1, \dots, R_m$. We then **prune** this tree, meaning that we collapse some of its leaves into the parent nodes

How should we decide how much to prune a tree?

Cross-validation

What is weird about applying this here?



For any tree T , let $|T|$ denote its number of leaves. We define

$$C_{\alpha}(T) = \sum_{j=1}^{|T|} [1 - \hat{p}_{c_j}(R_j)] + \alpha |T|$$

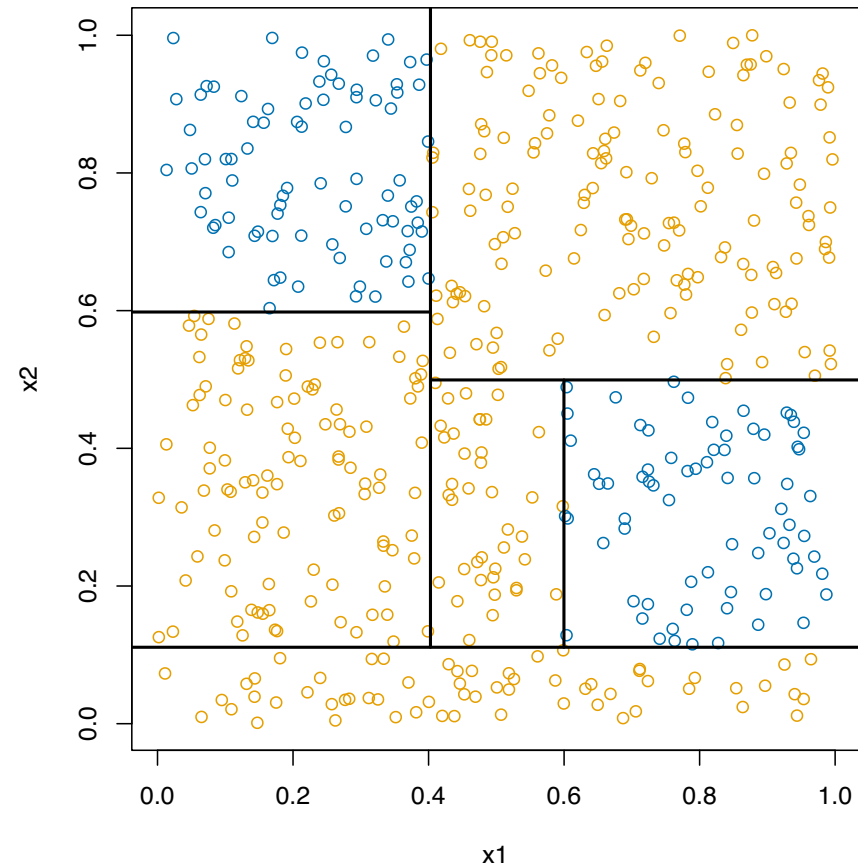
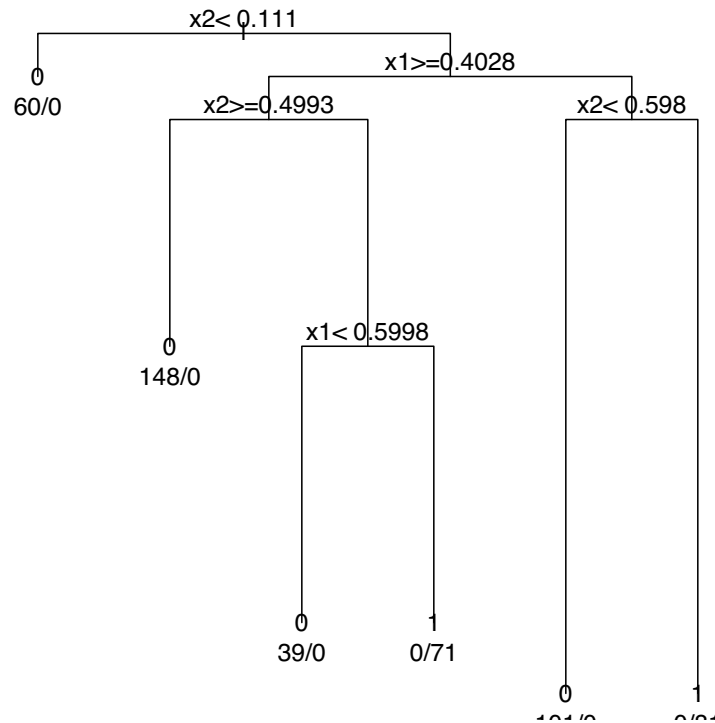
(Handwritten blue annotations: a circle around $C_{\alpha}(T)$, a circle around $|T|$ in the second term, and arrows pointing to the summation index j and the term $|T|$.)

We seek the tree $T \subseteq T_0$ that minimizes $C_{\alpha}(T)$. It turns out that this can be done by pruning the weakest leaf one at a time.

Note that α is a **tuning parameter**, and a larger α yields a smaller tree. CART picks α by 5- or 10-fold cross-validation

Example: simple classification tree

Example: $n = 500$, $p = 2$, and $K = 2$. We ran CART:



In R, can use rpart or tree. In python, DecisionTreeClassifier. However, python does not yet support pruning (as far as I can tell)!!